Alternative Assessment Test (AAT-2) Report on

“Execution of Combinational and Sequential Digital Modules on the Cadence Tool”

Submitted in partial fulfilment of the requirements of the Alternative Assessment Test (AAT) for the course

ADVANCED DIGITAL AND LOGIC DESIGN (ADLD)
[Course Code: 23ECSPE1AD]

Submitted by

Shivanshu Shukla **1BF24EC142**

Submitted to

Prof. Ajaykumar Devarapalli

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING
BMS College of Engineering
Bull Temple Road, Basavanagudi, Bengaluru-560019

Academic Year
2026 – 2027

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to everyone who helped and supported me in completing this Alternative Assessment Test (AAT-2) report on “Execution of Combinational and Sequential Digital Modules on the Cadence Tool”.

I am grateful to the Principal of BMS College of Engineering and to the Head of the Department of Electronics and Communication Engineering for providing the academic environment and the laboratory and software facilities, including the Cadence tools, needed to carry out this work.

I extend my heartfelt thanks to Prof. Ajaykumar Devarapalli, faculty of the course Advanced Digital and Logic Design (ADLD, 23ECSPE1AD), for his guidance, valuable suggestions and constant encouragement, which helped me understand how to design, compile and simulate digital circuits on the Cadence tool.

I also thank the teaching and non-teaching staff of the department, my friends for their support and discussions, and my family for their continuous motivation throughout this work.

Shivanshu Shukla
1BF24EC142

INDEX

INTRODUCTION.....2
Tools and Methodology.....2
Modules Covered.....2
Procedure on the Cadence Tool.....2
MODULE 1: 555 TIMER (COMBINATIONAL MODULE).....3
Design Description.....3
Circuit Diagram.....4
Verilog HDL Code.....4
Testbench.....5
Simulation Waveform.....5
MODULE 2: N-BIT COUNTER (SEQUENTIAL MODULE).....8
Design Description.....8
Circuit Diagram.....8
Verilog HDL Code.....9
Testbench.....9
Simulation Waveform.....10
CONCLUSION.....12

INTRODUCTION

This report presents the execution of one combinational module (555 timer) and one sequential module (N-bit counter) on the Cadence tool, prepared as part of the Alternative Assessment Test (AAT-2) for the course Advanced Digital and Logic Design (ADLD, 23ECSPE1AD). For each module the design description, circuit diagram, Verilog HDL code, testbench and simulation waveform are given.

Tools and Methodology

Each module is written in Verilog HDL using the gedit editor, together with its own testbench, on a Linux workstation. The design and testbench are compiled and simulated with Cadence NCVerilog (ncverilog +access+r), and the dumped .vcd file is opened in Cadence SimVision to observe and explain the waveforms. The sequential module is clocked with a 10 ns clock and all times are in ns (SimVision shows them in ps).

Modules Covered

No.	Module	Type	Design file	Testbench file
1	555 Timer	Combinational	timer555.v	tb_timer555.v
2	N-Bit Counter (N = 5)	Sequential	counter.v	tb_counter.v

Procedure on the Cadence Tool

The same flow was followed for both modules.

1. Log in with the student profile and open Home, then the Cadwork folder and the Digital folder.
2. Create a folder named with the USN (1BF24EC142); inside it create a folder sim, and inside sim create the folders rtl, tb and work.
3. Open a terminal in the Digital folder. Run csh and then source .cshrc to load the Cadence environment, and check the tool with which ncverilog.
4. Go to sim/rtl, write the design with gedit filename.v, save and quit. Run ncverilog +access+r filename.v to check the design for errors.
5. Go to ../tb and write the testbench with gedit tb_filename.v. It contains $dumpfile and $dumpvars so that a .vcd file is generated.
6. Go to ../work and run ncverilog +access+r ../rtl/filename.v ../tb/tb_filename.v. Use ls to confirm that the .vcd file is created.
7. Run simvision filename.vcd. In the left pane select the testbench, then click the small waveform icon at the top right to plot the signals.
8. Observe the waveform, demonstrate the working and explain the behaviour of the module.

MODULE 1: 555 TIMER (COMBINATIONAL MODULE)

Design Description

The 555 timer contains two comparators, an SR latch and a discharge transistor. This module models the control logic of the timer: the comparator outputs are the inputs TRIG_N (active low) and THRESH (active high), and RESET_N (active low) overrides everything. The module has no clock; the outputs follow the levels of the inputs through continuous assignments and a cross-coupled NAND pair. A low pulse on TRIG_N sets OUT high, OUT is held until THRESH goes high, and DISCH is the complement of OUT (the discharge transistor is ON when OUT is low).

Input / output ports

Port	Direction	Width	Description
TRIG_N	Input	1	Trigger, active low; a low level sets OUT = 1
THRESH	Input	1	Threshold, active high; a high level clears OUT to 0
RESET_N	Input	1	Reset, active low; forces OUT = 0 and overrides all inputs
OUT	Output	1	Timer output
DISCH	Output	1	Discharge transistor drive (1 = ON, OUT low)

Function table

RESET_N	TRIG_N	THRESH	OUT	DISCH	Action
0	X	X	0	1	Reset
1	0	X	1	0	Set
1	1	1	0	1	Clear
1	1	0	Hold	Hold	Hold

Circuit Diagram

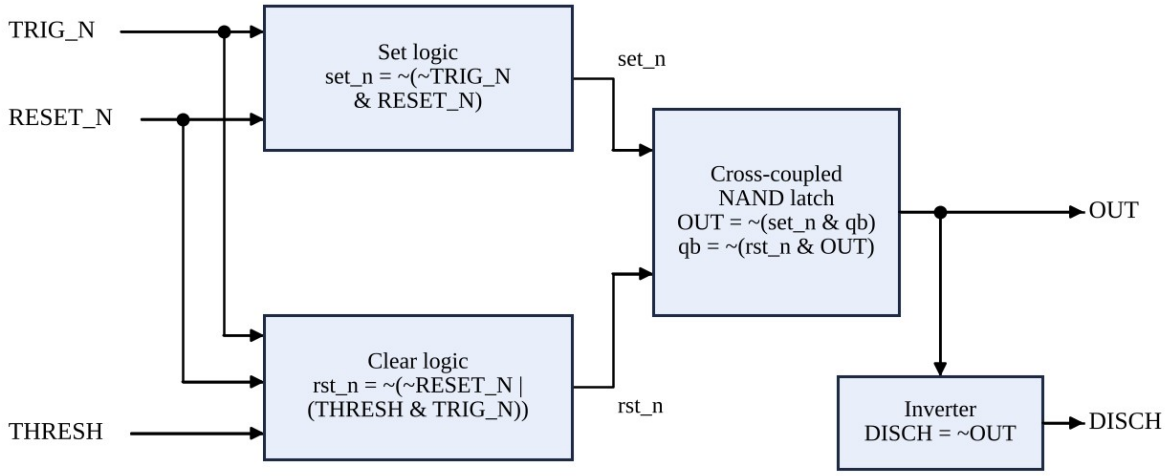


Figure 1.1: Block diagram of the 555 Timer

Verilog HDL Code

The design file timer555.v was written in gedit in the rtl folder.

Open

timer555.v

~/cadWorkDir/Digital/1BF24ECXXX/sim/rtl

```
//555 Timer - combinational (gate level) model
`timescale 1ns/1ps

module timer555 (TRIG_N, THRESH, RESET_N, OUT, DISCH);

input  TRIG_N;      // trigger   (active low)
input  THRESH;      // threshold (active high)
input  RESET_N;     // reset     (active low)
output OUT;         // timer output
output DISCH;       // discharge transistor (1 = ON)

wire set_n, rst_n, qb;

assign set_n = ~(~TRIG_N & RESET_N);          // set  condition
assign rst_n = ~(~RESET_N | (THRESH & TRIG_N)); // clear condition

assign OUT   = ~(set_n & qb);                 // cross coupled NAND pair
assign qb    = ~(rst_n & OUT);
assign DISCH = ~OUT;                          // ON when OUT low

endmodule
```

Figure 1.2: Verilog HDL code of the 555 Timer in gedit

Testbench

The testbench tb_timer555.v in the tb folder drives RESET_N, TRIG_N and THRESH in 10 ns steps and dumps the waveform to timer555.vcd.

Open

tb_timer555.v
~/cadWorkDir/Digital/1BF24ECXXX/sim/tb

```
//Test Bench for 555 timer
`timescale 1ns/1ps

module tb_timer555();

reg TRIG_N, THRESH, RESET_N;
wire OUT, DISCH;

timer555 DUT(.TRIG_N(TRIG_N), .THRESH(THRESH), .RESET_N(RESET_N),
.OUT(OUT), .DISCH(DISCH));

initial
begin
$dumpfile("timer555.vcd");
$dumpvars(0, tb_timer555);
end

initial
begin
TRIG_N=1; THRESH=0; RESET_N=0;
#10 RESET_N=1;
#10 TRIG_N=0;
#10 TRIG_N=1;
#10 THRESH=1;
#10 THRESH=0;
#10 TRIG_N=0;
#10 TRIG_N=1;
#10 RESET_N=0; TRIG_N=0;
#10 TRIG_N=1; RESET_N=1;
#20 $finish;
end

endmodule
```

Figure 1.3: Testbench of the 555 Timer in gedit

Simulation Waveform

The commands used in the terminal to compile, simulate and open the waveform:

Terminal - timer555

```
% cd ~/cadWorkDir/Digital/1BF24ECXXX/sim/rtl
% ncverilog +access+r timer555.v
% cd ../tb
% cd ../work
% ncverilog +access+r ../rtl/timer555.v ../tb/tb_timer555.v
% ls
% simvision timer555.vcd
```

Figure 1.4: Terminal commands used for the 555 Timer

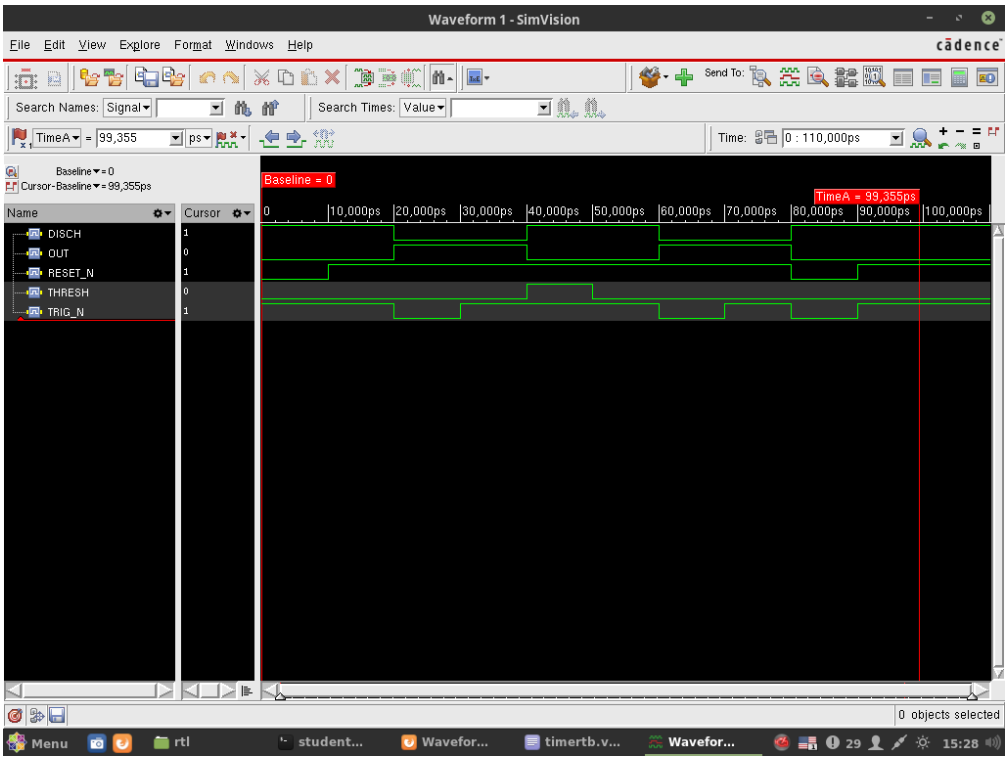


Figure 1.5: Simulation waveform of the 555 Timer in SimVision

Observation: OUT goes high when TRIG_N is pulsed low (at 20 ns and 60 ns) and stays high after TRIG_N returns high. It is cleared when THRESH goes high (at 40 ns). When RESET_N is low at 80 ns, OUT is forced low even though TRIG_N is also low, so reset has priority. DISCH is always the complement of OUT. With the cursor at 99,355 ps the values are DISCH = 1, OUT = 0, RESET_N = 1, THRESH = 0 and TRIG_N = 1.

Waveform behaviour

Time (ns)	Input event and behaviour	OUT	DISCH
0 - 10	RESET_N = 0: timer in reset, OUT forced low	0	1
10	RESET_N goes high; no trigger, OUT stays low	0	1
20	TRIG_N goes low: latch set, OUT high, discharge transistor off	1	0
30	TRIG_N returns high: OUT held high (latched)	1	0
40	THRESH goes high: latch cleared, OUT low, DISCH high	0	1
50	THRESH returns low: OUT stays low	0	1
60	TRIG_N goes low again: OUT set high	1	0
70	TRIG_N returns high: OUT held high	1	0
80 - 90	RESET_N and TRIG_N both low: reset has priority, OUT low	0	1
90 onwards	RESET_N and TRIG_N return high: OUT stays low	0	1

The expected timing diagram for the same testbench stimulus is shown below; it matches the waveform observed in SimVision.

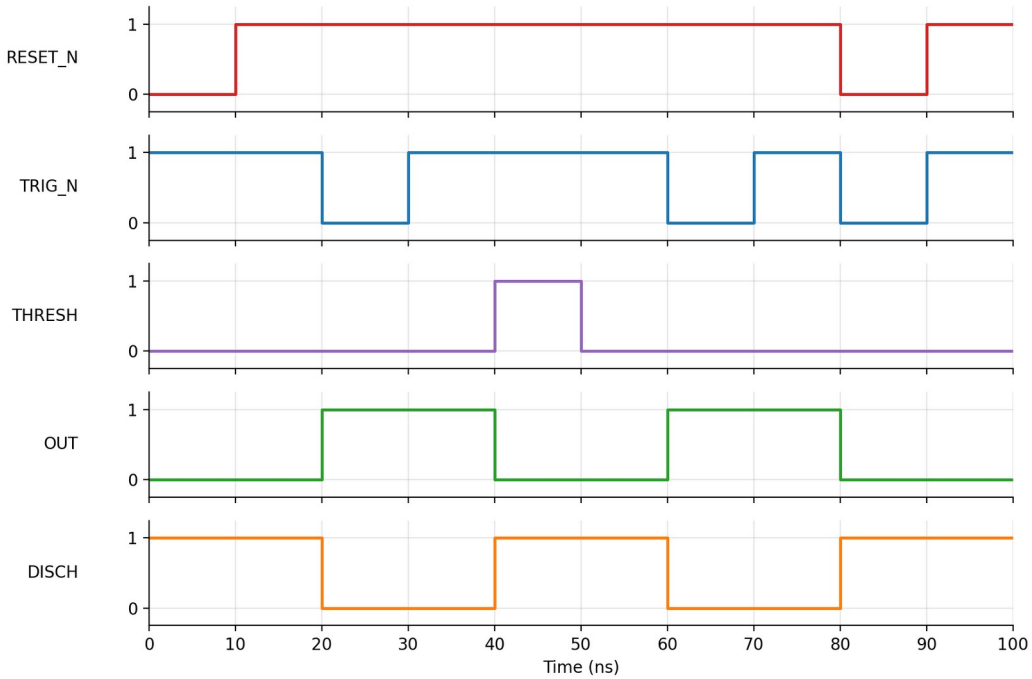


Figure 1.6: Expected timing diagram of the 555 Timer (plotted for the testbench stimulus)

MODULE 2: N-BIT COUNTER (SEQUENTIAL MODULE)

Design Description

The counter is a parameterized synchronous up-counter. On every rising edge of clk the count increases by one; when the count reaches N-1 it returns to 0, so the output repeats the sequence 0 to N-1. N is a parameter (N = 5 in the testbench, giving 0, 1, 2, 3, 4, 0, ...) and the output count[2:0] is 3 bits wide. An active-high reset rst clears the counter asynchronously.

Input / output ports

Port	Direction	Width	Description
N	Parameter	-	Count limit (count runs from 0 to N-1)
clk	Input	1	Clock, count updates on the rising edge
rst	Input	1	Asynchronous active-high reset
count[2:0]	Output	3	Counter value

Operation table

rst	Clock	Next count
1	X	0 (asynchronous reset)
0	Rising edge, count < N-1	count + 1
0	Rising edge, count = N-1	0 (wraps around)
0	No edge	No change

Circuit Diagram

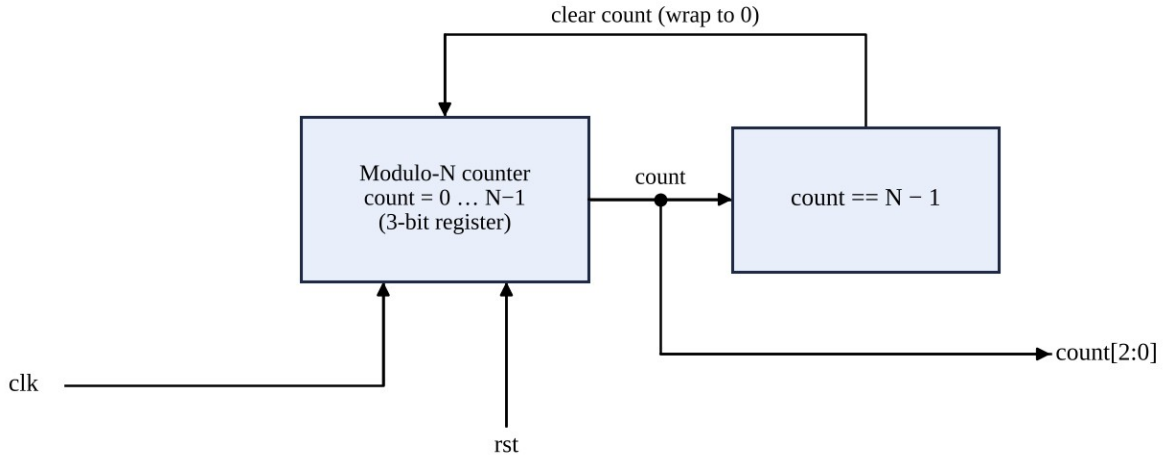


Figure 2.1: Block diagram of the N-Bit Counter

Verilog HDL Code

The design file counter.v was written in gedit in the rtl folder.

Open — counter.v
~/cadWorkDir/Digital/1BF24ECXXX/sim/rtl

```
//N-bit (modulo-N) up counter with asynchronous reset
`timescale 1ns/1ps

module counter #(parameter N = 5) (clk, rst, count);

input clk, rst;
output reg [2:0] count;

always @(posedge clk or posedge rst)
begin
if (rst)
count <= 0;
else if (count == N-1)
count <= 0;
else
count <= count + 1;
end

endmodule
```

Figure 2.2: Verilog HDL code of the N-Bit Counter in gedit

Testbench

The testbench tb_counter.v generates a 10 ns clock, applies reset from 0 to 12 ns and again from 120 to 130 ns, and ends the simulation at 170 ns.

Open — tb_counter.v
~/cadWorkDir/Digital/1BF24ECXXX/sim/tb

```
//Test Bench for N-bit counter
`timescale 1ns/1ps

module tb_counter();

parameter N = 5;
reg clk, rst;
wire [2:0] count;

counter #(.N(N)) DUT(.clk(clk), .rst(rst), .count(count));

initial clk = 0;
always #5 clk = ~clk;

initial
begin
$dumpfile("counter.vcd");
$dumpvars(0, tb_counter);
end

initial
begin
rst=1;
#12 rst=0;
#108 rst=1;
#10 rst=0;
#40 $finish;
end

endmodule
```

Figure 2.3: Testbench of the N-Bit Counter in gedit

Simulation Waveform

The commands used in the terminal to compile, simulate and open the waveform:

```
Terminal - counter

% cd ~/cadWorkDir/Digital/1BF24ECXXX/sim/rtl
% ncverilog +access+r counter.v
% cd ../tb
% cd ../work
% ncverilog +access+r ../rtl/counter.v ../tb/tb_counter.v
% ls
% simvision counter.vcd
```

Figure 2.4: Terminal commands used for the N-Bit Counter

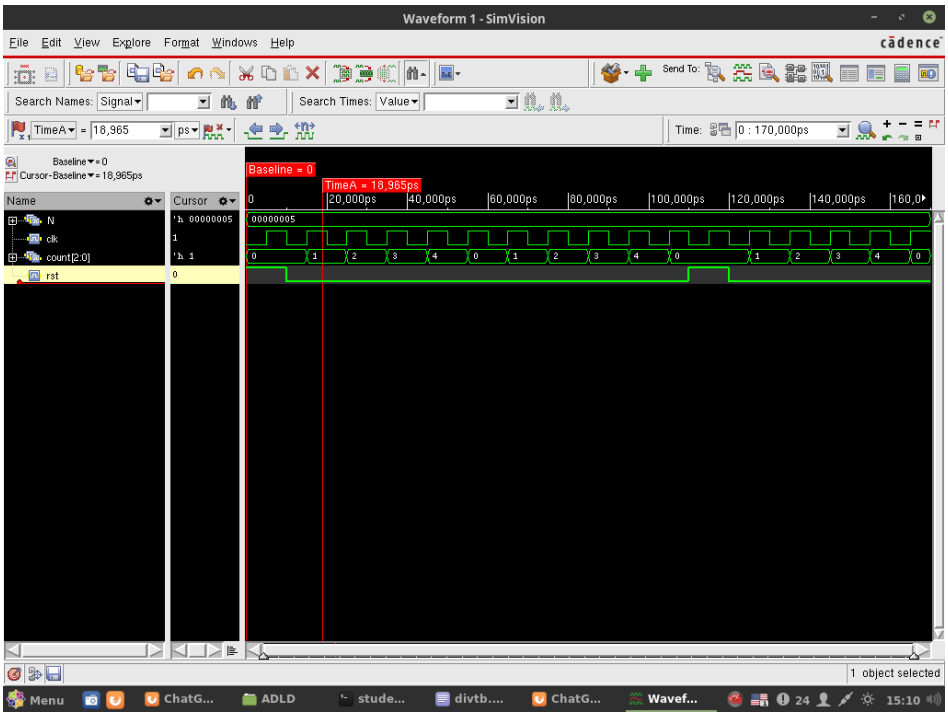


Figure 2.5: Simulation waveform of the N-Bit Counter in SimVision

Observation: The SimVision window shows N, clk, count[2:0] and rst of the testbench. N displays 00000005 (hex), confirming N = 5. At the cursor (TimeA = 18,965 ps) the reset is released (rst = 0), clk is high and count = 1, so the counter has started counting after the initial reset. count then follows 1, 2, 3, 4, 0, 1, ... on successive rising clock edges, and the second reset pulse clears it to 0 again, as shown in the expected timing diagram below.

Waveform behaviour

Event	count after the event	Remark
rst = 1 at start (0 - 12 ns)	0	Held in reset, clock edges ignored
rst released; 1st rising edge	1	Counting starts
2nd, 3rd, 4th rising edge	2, 3, 4	Increments by 1 every clock
5th rising edge (count = N-1)	0	Wraps around to 0
Following edges	1, 2, 3, 4, 0, ...	Sequence repeats (modulo-5)

Event	count after the event	Remark
rst = 1 in the middle of the run	0	Immediate clear, no clock edge needed
rst released again	1, 2, 3, 4 on next edges	Counting restarts from 0

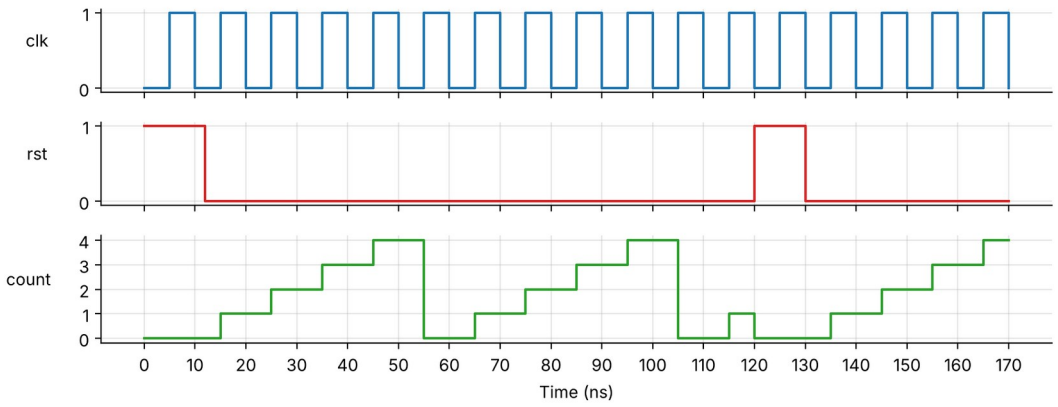


Figure 2.6: Expected timing diagram of the N-Bit Counter (plotted for the testbench stimulus)

CONCLUSION

One combinational module (555 timer) and one sequential module (N-bit counter) were written in Verilog HDL, compiled and simulated on the Cadence tool using NCVerilog, and their waveforms were analysed in SimVision. The 555 timer sets OUT on a low TRIG_N, clears it on a high THRESH, gives reset the highest priority and keeps DISCH as the complement of OUT. The N-bit counter counts 0 to N-1 on each rising clock edge, wraps around, and is cleared by rst. In both cases the simulated behaviour matched the expected function tables.

The exercise also covered the complete Cadence flow: creating the rtl, tb and work folders, setting up the environment with csh and .cshrc, compiling with ncverilog +access+r, generating the .vcd file and viewing it in SimVision.

Module	Type	Result observed in SimVision
555 Timer	Combinational	OUT set by low TRIG_N, cleared by high THRESH, forced low by RESET_N; DISCH = ~OUT
N-Bit Counter (N = 5)	Sequential	count runs 0, 1, 2, 3, 4, 0, ... on each rising clock edge; rst clears it to 0